FPGA-Oriented Evaluation of an LTE Turbo Decoder RTL

Course Project Report

Student Name(s) To Be Added
Department / Course / Institute To Be Added

***Abstract*—This report evaluates the current working LTE turbo-decoder RTL developed from the 2011 JSSC parallel radix-4 turbo-decoder architecture. The work progressed through four design phases: a baseline algorithmic model, a paper-aligned 8-lane parallel radix-4 architecture with QPP scheduling and batcher-based routing, a folded single-SISO FPGA rescue architecture, and the present rolling-window single-SISO implementation. Quantitative Vivado results on `xc7z010clg400-1` show that earlier folded revisions still exceeded the available LUT budget, while the final rolling-window RTL reduces synthesized LUT usage from $22\,282$ to 6418 and LUTRAM from 2336 to 22. Post-route implementation now fits the device, but misses a $100\,\mathrm{MHz}$ target with worst negative slack of $-23.738\,\mathrm{ns}$. MATLAB and GHDL-based BER experiments for $K = 3200$ and $K = 6144$ are also summarized; the RTL tracks the fixed-point radix-4/windowed model closely and remains within the same waterfall region as the floating-point reference.**

TABLE I
KEY NUMERIC CONFIGURATION USED IN THE CURRENT RTL

Item	Value
Constituent trellis states	8
Parallel paper target	$8\times$ SISO lanes
Current active top	$1\times$ SISO lane (folded)
Radix	Radix-4, two trellis steps per pair
Window length	30 bits = 15 radix-4 pairs
Maximum frame length	$K_{\max} = 6144$
Default half-iterations	11
Channel/parity LLR width	5-bit signed
Extrinsic width	6-bit signed
Posterior width	7-bit signed
State metric width	10-bit signed
Metric arithmetic	Fixed-width modulo-style add/sub/max
Extrinsic scaling	$11/16$ via shift-add
Interleaver law	LTE QPP table

I. DESIGN PROGRESSION

The starting point was a baseline LTE-style turbo decoding model: terminated 8-state RSC constituent encoders, QPP interleaving, iterative exchange of extrinsic information, and reference decoding in MATLAB using floating-point log-MAP / BCJR and floating max-log-MAP. This stage established the framing law, QPP parameters, LLR sign convention, and BER reference curves before hardware-specific compromises were introduced.

The second phase targeted the paper-aligned architecture of Studer *et al.* [1]. The saved parallel top in the repository, `turbo_decoder_top_parallel8_backup`, keeps the main architectural ingredients of that phase: $8\times$ SISO parallelism, radix-4 trellis processing, `qpp_parallel_scheduler`, `batcher_master`/`batcher_slave` routing, and banked `multiport_row_bram` memories. In that organization, the QPP scheduler produces lane-wise permutation indices, the batcher network sorts and unsorts lane requests to maintain contention-free routing, and the banked memories emulate the row-parallel data movement expected by the ASIC-oriented paper architecture.

The third phase was driven by FPGA infeasibility. Historical Vivado runs on the same class of target indicated that the paper-faithful parallel design was not practical on this device class; project notes recorded extreme LUT blow-up and poor BRAM inference. Consequently, the design was folded to a single active SISO decoder, scalarized the QPP access, and replaced the full multi-lane scheduling/routing fabric with a monolithic top-level FSM and BRAM-backed frame memories.

The current fourth phase is the working RTL used in this report. It still preserves the core max-log-MAP/radix-4 decoding math, but the microarchitecture of `siso_maxlogmap` was rewritten to a rolling-window schedule. Each $30\,\mathrm{bit}$ window is processed by: 1) forward recursion on the current window while caching local α, g_0, g_1, and γ, 2) dummy backward recursion on the next window to generate the current-window β seed, and 3) local backward recursion and LLR extraction on the current window. This removes the earlier full-segment alpha-seed prepass and also avoids branch/gamma recomputation during LLR extraction.

II. CURRENT RTL AND NUMERIC ASSUMPTIONS

All results reported here refer to the current VHDL RTL synthesized for `xc7z010clg400-1`. The active top is `turbo_decoder_top`; it instantiates one `siso_maxlogmap` with external pair fetch enabled, 16 BRAM-backed channel/extrinsic/posterior memories, and an incremental QPP recurrence inside the top FSM.

The simulation and evaluation data are also explicit. The RTL testbench consumes `lte_frame_input_vectors.txt`, whose first row is $\{K, n_{\text{half-iter}}, f_1, f_2\}$, followed by rows `idx bit_orig bit_int` $l_{\mathtt{sys}}$ $l_{\mathtt{par1}}$ $l_{\mathtt{par2,int}}$. The main RTL output used for BER is `tb_turbo_top_final_llrs.txt` with rows `idx_orig final_llr seen`; the hard decision rule is 1

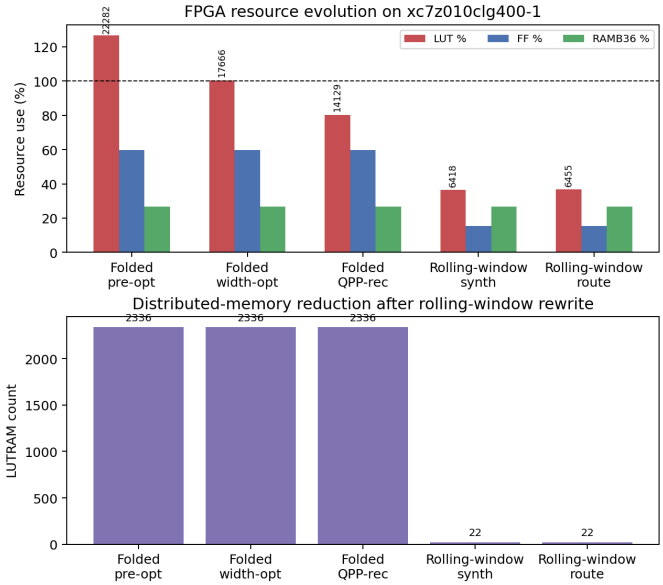


Fig. 1. Evolution of FPGA resource usage across the main folded RTL revisions. The rolling-window rewrite removes most LUTRAM pressure while preserving the same BRAM budget.

if final LLR > 0, else 0. Internally, the MATLAB and Python reference flows operate on the matrices $\mathbf{L}_{nat}^{sys}$, $\mathbf{L}_{nat}^{par1}$, $\mathbf{L}_{int}^{par2}$, $\mathbf{L}_{nat/int}^{apri}$, $\mathbf{L}_{nat/int}^{ext}$, $\mathbf{L}_{nat/int}^{post}$, and the QPP permutation vector π.

III. FPGA Synthesis and Implementation Results

Table II summarizes the architectural evolution observed in Vivado. The earliest folded single-SISO revision still over-utilized LUTs by 26.6% (22 282/17 600). A width-reduced revision lowered this to 17 666 LUTs, and an incremental-QPP revision reduced it further to 14 129 LUTs. The final rolling-window rewrite is the first version that fits cleanly in synthesis and in post-route implementation.

The main inference from Table II is not only the LUT reduction, but the near elimination of distributed frame memory. Earlier versions carried 2336 LUTRAMs because several SISO memories did not map cleanly to BRAM. In the current implementation, only small local memories remain as LUTRAM (22 total), while the top-level frame stores map to 16 RAMB36 blocks. The hierarchy report attributes 5948 LUTs of the final synthesized design to `siso_u`, confirming that the constituent decoder remains the dominant hardware cost.

The final synthesis log also shows that only 7 warnings remain, dominated by removed legacy preload memories and one floorplanning warning. However, implementation timing is still the open problem. At the current 100 MHz clock constraint, the routed design reports worst negative slack of -23.738 ns, 5140 setup failing endpoints, and total setup violation of $-16\,900.556$ ns. Thus, the design is area-feasible but not yet frequency-closed at 100 MHz; a slower clock or further pipelining is required for hardware deployment.

IV. MATLAB, RTL, and BER Evaluation

The MATLAB environment provides three decoder models: 1) ideal floating log-MAP / BCJR, 2) floating max-log-MAP, and 3) an RTL-style fixed-point radix-4/windowed model that mirrors the active hardware assumptions in Table I. For $K = 3200$, the heavy MATLAB sweep uses 24 frames (76 800 bits total per SNR point); for $K = 6144$, it uses 16 frames (98 304 bits total per SNR point). The RTL BER sweep is not predicted statistically; it is measured from actual GHDL simulation using `tools/run_ber_sweep.py`, which generates AWGN-corrupted input vectors, runs `tb_turbo_top`, and computes BER from `tb_turbo_top_final_llrs.txt`. The current saved RTL sweep aggregates 8 simulated frames per SNR point for both $K = 3200$ and $K = 6144$.

The BER figures show three useful trends. First, the fixed-point radix-4/windowed MATLAB model is predictably worse than the ideal and floating references because of quantization, extrinsic scaling, and windowing. Second, the actual RTL BER remains close to the fixed-point model, which validates the present RTL numerics. Third, the longer $K = 6144$ frame shows better BER than $K = 3200$ for the same SNR range, consistent with turbo-code waterfall behavior.

At 2.5 dB, the $K = 3200$ BER values are 7.29×10^{-4} (ideal), 7.16×10^{-4} (floating max-log), 5.34×10^{-4} (fixed), and 4.69×10^{-4} (RTL). For $K = 6144$, the corresponding values are 3.05×10^{-4}, 3.15×10^{-4}, 2.85×10^{-4}, and 3.87×10^{-4}, respectively. A deterministic end-to-end regression at $K = 6144$, $f_1 = 263$, $f_2 = 480$, and 11 half-iterations also reports zero bit errors against the original transmitted bits in `tb_turbo_top_report.txt`, confirming correct frame-level integration for at least one clean test vector.

V. Conclusion

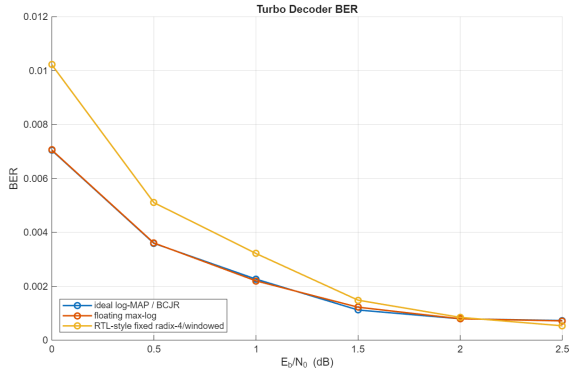
This project reached a working FPGA-oriented turbo-decoder RTL by progressively sacrificing ASIC-style parallelism for FPGA realizability. The paper-aligned architecture remains important as the throughput target and as the conceptual source of the QPP scheduling, batcher routing, and radix-4 SISO approach. Nevertheless, the current `xc7z010clg400-1` target cannot absorb that architecture efficiently. The present rolling-window single-SISO RTL is the first version that fits after synthesis and route, with a large reduction in LUT and LUTRAM usage while maintaining BER behavior close to the fixed-point reference model. The remaining engineering gap is timing closure at 100 MHz; therefore, future work should prioritize pipeline insertion, BRAM output registering, and board-oriented wrapper integration rather than further increasing parallelism on this device.

References

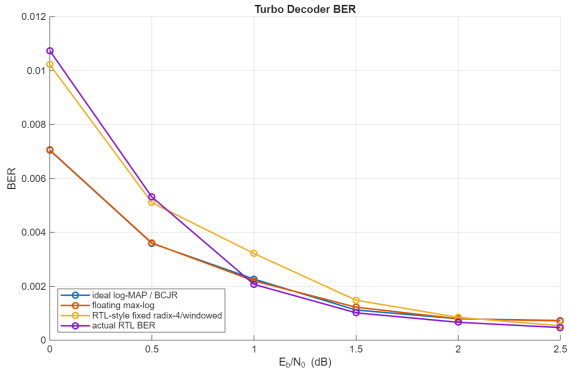
[1] C. Studer, C. Benkeser, S. Belfanti, and Q. Huang, "Design and implementation of a parallel turbo-decoder ASIC for 3GPP-LTE," *IEEE Journal of Solid-State Circuits*, 2011.

TABLE II
VIVADO RESOURCE EVOLUTION ON `xc7z010clg400-1`

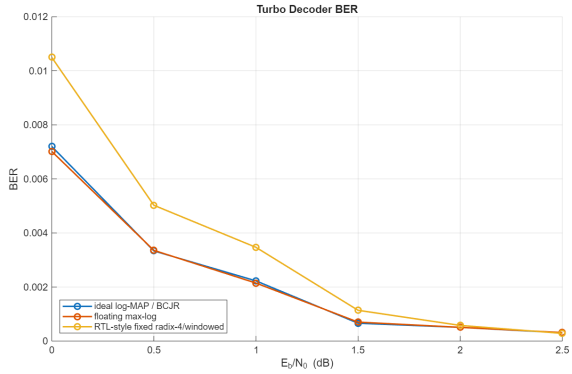
Revision / Report Source	LUTs	LUT as Logic	LUTRAM	FFs	RAMB36	
Folded pre-optimization (`utilization_report_tmp.txt`)	22282	19946	2336	21079	16	Ov
Folded width-optimized (`utilization_report_widthopt_tmp.txt`)	17666	15330	2336	21065	16	Still
Folded with QPP recurrence (`utilization_report.txt`)	14129	11793	2336	21080	16	Fits in syn
Current rolling-window synth (`utilization_qpprec_tmp.txt`)	6418	6396	22	5410	16	
Current rolling-window route (`implementation_utilization_qpprec_tmp.txt`)	6455	6433	22	5389	16	



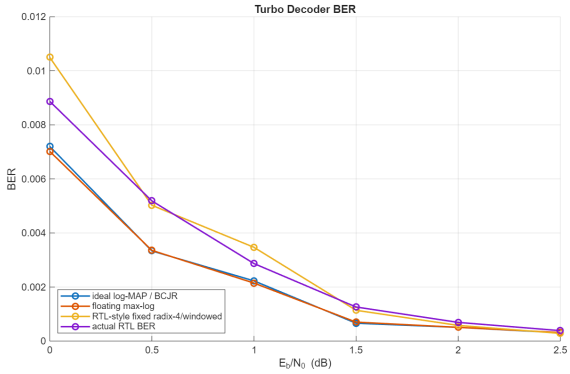
(a) $K = 3200$: MATLAB BER curves only



(b) $K = 3200$: MATLAB vs. actual RTL



(c) $K = 6144$: MATLAB BER curves only



(d) $K = 6144$: MATLAB vs. actual RTL

Fig. 2. All currently available performance/evaluation plots used in this project. The filenames retain `ber_fer` from an earlier plotting flow, but the generated plots are BER-only.